软件性能工程——Project 1 测试版 write-up

邓林 | 2026 年春季学期

范浩、李晓慧 | 2026 年 03 月 24 日

一、耗时表

设计	编码	调试	测试	会议
2h	1.5h	2h	3.5h	2h

二、问题重述

本项目的核心任务是优化 `bitarray.c` 中的 `bitarray_rotate` 函数，该函数用于将一个大型的位数组（Bit Array）向左或向右旋转指定的位数，其中子数组的起始索引、长度和位移量都是输入的参数。
原实现的核心是 `bitarray_rotate()` 、 `bitarray_rotate_left()` 和 `bitarray_rotate_left_one()` 三个函数。原始代码的实现是通过循环调用单次左移（`bitarray_rotate_left_one`）来完成旋转，时间复杂度高达 $O(N \times K)$（N 为数组长度，K 为旋转位数）。在处理大数据量时，性能极差。我们要做的就是在不违反程序正确性的前提下，提高程序的性能。

三、使用`perf`工具分析程序及确定优化点的过程

1、代码结构：

根据对测试流程的跟踪，代码主要的执行路径为：
1. 入口文件：`main.c -> main()`
2. 解析输入选项（`n, t, s, m, l`），对应不同的 `timed_rotation()` 时间限制
3. `timed_rotation()` -> 生成 `bit_offset`，`bit_right_shift_amount`，`bit_length`，`bit_sz`
4. 核心调用：`testutil_rotate() -> bitarray_rotate() -> bitarray_rotate_left() -> bitarray_rotate_left_one()`

2、诊断瓶颈

```
Samples: 7K of event 'cpu-clock:pppH', Event count (approx.): 1856500000
  Children      Self  Command   Shared Object       Symbol
+   97.20%     0.00%  everybit  libc-2.31.so        [.] __libc_start_main
+   97.19%     0.00%  everybit  everybit            [.] main
+   97.16%     0.00%  everybit  everybit            [.] timed_rotation
+   97.16%     0.00%  everybit  everybit            [.] testutil_rotate
+   97.16%     0.00%  everybit  everybit            [.] bitarray_rotate
+   97.15%     0.05%  everybit  everybit            [.] bitarray_rotate_left
+   95.38%    14.21%  everybit  everybit            [.] bitarray_rotate_left_one
+   54.59%    46.19%  everybit  everybit            [.] bitarray_set
+   22.18%    18.53%  everybit  everybit            [.] bitarray_get
+   18.07%    18.06%  everybit  everybit            [.] bitmask
+    0.90%     0.00%  everybit  [kernel.kallsyms]   [k] entry_SYSCALL_64_after_hwframe
+    0.90%     0.00%  everybit  [kernel.kallsyms]   [k] do_syscall_64
+    0.86%     0.00%  everybit  [kernel.kallsyms]   [k] x64_sys_call
+    0.53%     0.00%  everybit  [unknown]           [.] 0x41e589480000b274
     0.50%     0.00%  everybit  libc-2.31.so        [.] clock_gettime@GLIBC_2.2.5
     0.50%     0.22%  everybit  [vdso]              [.] __vdso_clock_gettime
     0.38%     0.15%  everybit  ld-2.31.so          [.] _dl_sysdep_start
     0.34%     0.34%  everybit  [kernel.kallsyms]   [k] __lock_text_start
     0.31%     0.18%  everybit  ld-2.31.so          [.] mmap64
     0.31%     0.03%  everybit  libc-2.31.so        [.] __GI___libc_write
     0.31%     0.03%  everybit  ld-2.31.so          [.] _dl_map_object
     0.28%     0.28%  everybit  [kernel.kallsyms]   [k] queue_work_on
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] __x64_sys_write
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] ksys_write
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] vfs_write
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] __x64_sys_clock_gettime
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] new_sync_write
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] process_cpu_clock_get
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] tty_write
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] file_tty_write.isra.0
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] posix_cpu_clock_get
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] cpu_clock_sample_group
     0.28%     0.00%  everybit  [kernel.kallsyms]   [k] n_tty_write
```

```
xxyhh@ubuntu:~/Desktop/project1/everybit$ sudo perf record -g ./everybit -l
[sudo] password for xxyhh:
---- RESULTS ----
Tier 0 (≈0B) completed in 0.000304s
Tier 1 (≈0B) completed in 0.000086s
Tier 2 (≈1B) completed in 0.000320s
Tier 3 (≈1B) completed in 0.000097s
Tier 4 (≈2B) completed in 0.000353s
Tier 5 (≈4B) completed in 0.000253s
Tier 6 (≈6B) completed in 0.000357s
Tier 7 (≈11B) completed in 0.000418s
Tier 8 (≈18B) completed in 0.001569s
Tier 9 (≈29B) completed in 0.002658s
Tier 10 (≈47B) completed in 0.015751s
Tier 11 (≈76B) completed in 0.035051s
Tier 12 (≈123B) completed in 0.068812s
Tier 13 (≈199B) completed in 0.200193s
Tier 14 (≈323B) completed in 0.570529s
Tier 15 (≈522B) exceeded 1.00s cutoff with time 1.481421s
Succesfully completed tier: 14
---- END RESULTS ----
[ perf record: Woken up 3 times to write data ]
[ perf record: Captured and wrote 0.858 MB perf.data (7426 samples) ]
```

通过对原始代码的 `perf` 分析发现，程序的瓶颈主要集中在 `bitarray_rotate()` 的实现逻辑上。原始代码使用 `bitarray_rotate_left_one()` 函数来实现每次对子数组左移一位。这种方式虽然保证了零额外的空间开销，但它的时间复杂度高达 $O(N \times K)$（其中 N 是子数组的长度，K 是左移的位数）。当处理大数据量（如超大型位数组）时，会产生数以亿计的逐位读写，引发极其严重的性能瓶颈。

3、优化措施

核心思路：**“空间换时间”**。 将需要移动的前面部分子数组（长度为 K）提取并复制到一个临时分配的缓冲区中，接着将后面的子数组（长度为 N-K）直接整体平移到目标位置，最后再将缓冲区中的内容一次性补到末尾。。

四、优化思路

在确立算法时，我们对比了以下三种经典方法：

1. **缓冲交换法**：将 A 复制到中间数组 tmp，移动 B 到最终位置，再从 tmp 中取出数组放到 A 的最终位置。
   - *优点*：位移动次数少，逻辑简单。
   - *缺点*：若目标字符串较长，tmp 空间过大，会影响缓存性能。
2. **环状替换法**：创建单次循环左移字符，旋转次数取数组长度的模数作为循环次数。
   - *优点*：bit 移动次数最小化，仅使用常数级临时空间。
   - *缺点*：内存访问极其分散，对 CPU 缓存非常不利。
3. **三重翻转法**：$(A^R B^R)^R = BA$
   - *优点*：In-place 操作，取逆只需使用常量存储。
   - *缺点*：读写次数较多（约 $4N$ 次），常数开销大。

根据老师在project 1的文档中给出的提示，基于工程实现的稳健性与底层读写次数的考量，我们选择了**第一种方法（缓冲交换法）**。为了解决临时数组的开销与安全性问题，我们进行了以下设计权衡：

动态堆空间分配（`malloc`）的纯粹实现策略： 我们的代码没有采用复杂的位压缩缓冲区，而是直接使用 `malloc` 开辟了一个大小为 `K` 的 `bool` 类型数组（`bool* tmp = (bool*)malloc(sizeof(bool) * k);`）。

- **优势**：这种设计完美契合了底层提供的 `bitarray_get` 和 `bitarray_set` 接口。无需在缓冲区内部再次处理繁琐的位运算（除法和取模），大大降低了代码的复杂度和出错率；同时，使用堆空间彻底避免了使用 VLA（变长数组）在处理大数组时可能引发的栈溢出。
- **代价与权衡**：虽然 `malloc` 会带来系统调用的开销，并且 `bool` 数组未进行位级压缩导致一定的内存放大，但在第一阶段 $O(N)$的优化中，**减少底层 `get/set` 的调用总次数（从 4N 降到 N+K）带来的性能收益，远大于内存分配的损耗。**

优化效果：
通过上述优化措施，我们将算法的时间复杂度从 $O(N \times K)$ 骤降到了线性的 $O(N)$，极大地提高了程序的执行性能（在 1 秒内可处理的数据量由几十 KB 跃升至约 1MB，即 Tier 32），并且完美通过了所有默认测试集以及我们自定义的极端边界测试用例。

```
xxyhh@ubuntu:~/Desktop/project1/everybit$ ./everybit -t tests/default
Testing file tests/default.

Running test #0...
 --> tests/default at line 34: PASS

Running test #1...
 --> tests/default at line 41: PASS

Running test #2...
 --> tests/default at line 49: PASS
 --> tests/default at line 52: PASS
 --> tests/default at line 55: PASS
 --> tests/default at line 58: PASS
 --> tests/default at line 61: PASS
Done testing file tests/default.
```

```
Running test #19...
 --> tests/mytests at line 160: PASS

Running test #20...
 --> tests/mytests at line 166: PASS

Running test #21...
 --> tests/mytests at line 172: PASS

Running test #22...
 --> tests/mytests at line 180: PASS

Running test #23...
 --> tests/mytests at line 186: PASS

Running test #24...
 --> tests/mytests at line 194: PASS

Running test #25...
 --> tests/mytests at line 200: PASS

Running test #26...
 --> tests/mytests at line 208: PASS
Done testing file tests/mytests.
```

```
xxyhh@ubuntu:~/Desktop/project1/everybit$ ./everybit -l
---- RESULTS ----
Tier 0 (≈0B) completed in 0.000092s
Tier 1 (≈0B) completed in 0.000087s
Tier 2 (≈1B) completed in 0.000092s
Tier 3 (≈1B) completed in 0.000087s
Tier 4 (≈2B) completed in 0.000087s
Tier 5 (≈4B) completed in 0.000089s
Tier 6 (≈6B) completed in 0.000091s
Tier 7 (≈11B) completed in 0.000093s
Tier 8 (≈18B) completed in 0.000096s
Tier 9 (≈29B) completed in 0.000375s
Tier 10 (≈47B) completed in 0.000110s
Tier 11 (≈76B) completed in 0.000118s
Tier 12 (≈123B) completed in 0.000134s
Tier 13 (≈199B) completed in 0.000230s
Tier 14 (≈323B) completed in 0.000206s
Tier 15 (≈522B) completed in 0.000414s
Tier 16 (≈845B) completed in 0.000393s
Tier 17 (≈1KB) completed in 0.001025s
Tier 18 (≈2KB) completed in 0.000935s
Tier 19 (≈3KB) completed in 0.002103s
Tier 20 (≈5KB) completed in 0.002428s
Tier 21 (≈9KB) completed in 0.004124s
Tier 22 (≈14KB) completed in 0.006482s
Tier 23 (≈23KB) completed in 0.008121s
Tier 24 (≈38KB) completed in 0.013785s
Tier 25 (≈62KB) completed in 0.027156s
Tier 26 (≈101KB) completed in 0.045266s
Tier 27 (≈164KB) completed in 0.073067s
Tier 28 (≈265KB) completed in 0.121675s
Tier 29 (≈430KB) completed in 0.199553s
Tier 30 (≈696KB) completed in 0.273500s
Tier 31 (≈1MB) completed in 0.426079s
Tier 32 (≈1MB) completed in 0.805639s
Tier 33 (≈2MB) exceeded 1.00s cutoff with time 1.405851s
Succesfully completed tier: 32
---- END RESULTS ----
```

五、测试用例设计

为了保证代码的鲁棒性，并在互评环节获取高覆盖率分数，我们设计了 26 个白盒与黑盒测试用例。测试用例的设计不仅覆盖了规格本身，还兼顾了“朴素搬移 / 环状替换/ 三重翻转 / 64位块级并行”等多种潜在实现策略的缺陷漏洞。具体分类如下：

1. **基础逻辑与无用功过滤 (TC 1-4)**：
   测试了 `bit_length = 0`、`bit_length = 1`、旋转量为 0，以及旋转量为数组长度整数倍（`amount % length = 0`）的情况。用于验证程序是否能正确捕获这些状态并直接返回，避免除零错误或执行无用计算。
2. **大跨度取模与负数支持 (TC 9-10)**：
   设计了如 `801 % 8` 和 `-801 % 8` 的用例，强制验证自定义 `modulo` 函数对于大整数越界和 C 语言中负数取模特性的处理是否正确。
3. **算法实现陷阱与 GCD 边界 (TC 17-18, TC 21)**：
   针对“环状替换法”，设计了长度与旋转量互质（GCD=1）和非互质（GCD>1）的用例；针对“缓冲交换法”，设计了子块长度恰好等于旋转量两倍（$length = 2 \times amount$）的死锁陷阱，确保所有 $O(N)$ 算法均不产生遗漏。
4. **内存对齐与字边界破坏 (TC 7, 15-16, 19-20)**：
   此类用例专门用于攻击那些进行了“字级并行（Word-level parallel）”优化的代码。测试了非字节对齐偏移（如 offset=1 或 63）、跨越单字节、64位边界甚至 128 位边界的移位操作，验证子区间边界处的位掩码（Bitmask）操作是否发生越界污染。
5. **模式保留与区间隔离 (TC 8, 11-14, 24-26)**：
   使用全 0、全 1、交替模式（`101010`）、孤立点以及超长的 128 位数据块进行旋转。重点验证在旋转数组中间的子区间时，区间外部的数据未受到任何干扰和污染。

六、代码实现

```
/**
 * 将 a 复制到中间数组 tmp，移动 b 到最终位置，再从 tmp 中取出数组放到 a 的最终位置。
 * a 是指从 bit_offset 开始、长度为 bit_left_amount 的位子串。
 * b 是指紧随其后、长度为 bit_length - bit_left_amount 的位子串。
 */
static void bitarray_rotate_left(bitarray_t* const bitarray,
                                 const size_t bit_offset,
                                 const size_t bit_length,
                                 const size_t bit_left_amount) {
  // 处理特殊情况：旋转量为0或长度为0
  if (bit_length == 0) return;
  size_t k = bit_left_amount % bit_length;
  if (k == 0) return;

  size_t n = bit_length;

  // 1. 分配临时空间存储子串 a (长度为 k)
  // 这里使用 bool 数组最为直观，也可以使用 bitarray_new 分配位数组
  bool* tmp = (bool*)malloc(sizeof(bool) * k);
  if (tmp == NULL) {
    return; // 实际应用中应有错误处理
  }

  // 2. 将子串 a 复制到 tmp
  // a 的范围：[bit_offset, bit_offset + k)
  for (size_t i = 0; i < k; i++) {
    tmp[i] = bitarray_get(bitarray, bit_offset + i);
  }

  // 3. 移动子串 b 到最终位置（即向左平移 k 位）
```

```
  // b 的范围：[bit_offset + k, bit_offset + n)
  // 移动后 b 的新范围：[bit_offset, bit_offset + n - k)
  for (size_t i = 0; i < n - k; i++) {
    bool bit_val = bitarray_get(bitarray, bit_offset + k + i);
    bitarray_set(bitarray, bit_offset + i, bit_val);
  }

  // 4. 从 tmp 中取出子串 a，放到最终位置（紧跟在 b 后面）
  // a 的新范围：[bit_offset + n - k, bit_offset + n)
  for (size_t i = 0; i < k; i++) {
    bitarray_set(bitarray, bit_offset + (n - k) + i, tmp[i]);
  }

  // 5. 释放临时空间
  free(tmp);
}
```

七、编程过程中遇到的 bug 或陷阱

在项目的推进过程中，我们主要解决了以下两个典型的工程陷阱：

1. **栈溢出陷阱**： 在最初构思“临时缓冲区法”时，为了编写方便，我们曾尝试使用 C99 的变长数组如 `bool tmp[k];` 来代替 `malloc`。在面对小型测试用例时一切正常，但在运行性能测试（处理长达数 MB 的位数组）时，程序瞬间抛出段错误崩溃。分析发现，由于 VLA 在系统栈上分配内存，而 Linux 默认栈大小有限（通常为 8MB），超大数组直接导致了栈穿透。这促使我们摒弃 VLA，最终采用了更稳健的堆分配 `malloc` 方案。
2. **跨平台换行符冲突 导致解析崩溃**： 在编写和导入自定义测试用例（`tests/mytests`）时，终端在运行测试的第一秒就抛出了 `Assertion c == '0' || c == '1' failed` 的严重错误并导致 Core Dump。经过排查发现，这是因为测试脚本文件在 Windows 环境下编辑过，系统自动引入了 `\r\n` (CRLF) 换行符。当该文件被 Linux 虚拟机中的 C 语言评测框架读取时，隐藏的回车符 `\r` 触发了格式断言失败。最终我们通过在终端使用 `dos2unix` 工具批量转换文件格式为纯 LF，成功清除了不可见字符，使 26 个测试用例顺利跑通。

```
xxyhh@ubuntu:~/Desktop/project1/everybit$ ./everybit -t tests/mytests
Testing file tests/mytests.
Unknown command
Unknown command
Unknown command
Unknown command

Running test #1...
everybit: tests.c:323: _Bool boolfromchar(const char): Assertion `c == '0' || c == '1'' failed.
Aborted (core dumped)
xxyhh@ubuntu:~/Desktop/project1/everybit$ dos2unix tests/mytests
dos2unix: converting file tests/mytests to Unix format...
```

八、其他信息："无用功"——三重翻转法的尝试与性能对比对比

在确定最终使用“动态缓冲区法”之前，为了追求空间复杂度 $O(1)$，**我们还完整实现并测试了另一种经典算法：三重翻转法。**

1、算法思想

三重翻转法的数学原理为 $(A^R B^R)^R = BA$。
即将数组分为要移动的部分 A 和剩余部分 B:

1. 翻转 A 区间;
2. 翻转 B 区间;
3. 翻转整体 $(A + B)$ 区间。

为了实现这一过程，我们额外编写了 `bitarray_reverse` 辅助函数，采用首尾双指针向中间靠拢的方式进行逐位交换，并加入了一个**微小但关键的优化**：只有当 `left_bit != right_bit` 时才执行 `bitarray_set`，从而减少了一半的不必要内存写入。

```
static void bitarray_reverse(bitarray_t* const bitarray,
                             const size_t bit_offset,
                             const size_t bit_length) {
    if (bit_length <= 1) return;
    size_t left = bit_offset;
    size_t right = bit_offset + bit_length - 1;

    while (left < right) {
        bool left_bit = bitarray_get(bitarray, left);
        bool right_bit = bitarray_get(bitarray, right);

        // 微优化：仅在位不同时才进行写入，降低写开销
        if (left_bit != right_bit) {
            bitarray_set(bitarray, left, right_bit);
            bitarray_set(bitarray, right, left_bit);
        }
        left++;
        right--;
    }
}
```

2、性能测试

理论上，三重翻转法完全不需要 `malloc` 动态分配内存，不仅没有内存开销，还彻底杜绝了栈溢出的风险。但经过实际的长耗时测试，**该方法的最高成绩止步于 Tier 31 (~1MB)，略逊于缓冲区法的 Tier 32。**

```
xxyhh@ubuntu:~/Desktop/project1/everybit$ ./everybit -l
---- RESULTS ----
Tier 0 (≈0B) completed in 0.000088s
Tier 1 (≈0B) completed in 0.000088s
Tier 2 (≈1B) completed in 0.000087s
Tier 3 (≈1B) completed in 0.000087s
Tier 4 (≈2B) completed in 0.000494s
Tier 5 (≈4B) completed in 0.000094s
Tier 6 (≈6B) completed in 0.000092s
Tier 7 (≈11B) completed in 0.000093s
Tier 8 (≈18B) completed in 0.000098s
Tier 9 (≈29B) completed in 0.000331s
Tier 10 (≈47B) completed in 0.000037s
Tier 11 (≈76B) completed in 0.000044s
Tier 12 (≈123B) completed in 0.000095s
Tier 13 (≈199B) completed in 0.000065s
Tier 14 (≈323B) completed in 0.000111s
Tier 15 (≈522B) completed in 0.000122s
Tier 16 (≈845B) completed in 0.000533s
Tier 17 (≈1KB) completed in 0.000923s
Tier 18 (≈2KB) completed in 0.001359s
Tier 19 (≈3KB) completed in 0.002343s
Tier 20 (≈5KB) completed in 0.003985s
Tier 21 (≈9KB) completed in 0.005944s
Tier 22 (≈14KB) completed in 0.009663s
Tier 23 (≈23KB) completed in 0.015054s
Tier 24 (≈38KB) completed in 0.022685s
Tier 25 (≈62KB) completed in 0.040721s
Tier 26 (≈101KB) completed in 0.066156s
Tier 27 (≈164KB) completed in 0.100420s
Tier 28 (≈265KB) completed in 0.158357s
Tier 29 (≈430KB) completed in 0.271386s
Tier 30 (≈696KB) completed in 0.459393s
Tier 31 (≈1MB) completed in 0.724170s
Tier 32 (≈1MB) exceeded 1.00s cutoff with time 1.118919s
Succesfully completed tier: 31
---- END RESULTS ----
```

基于数据的瓶颈剖析：
在使用 `perf` 工具分析后我们得知，底层的 `bitarray_get` 和 `bitarray_set` 函数内部包含了昂贵的除法（`/ 8`）和取模（`% 8`）运算。**在巨大的常数倍数（$4N$ 对比 $N+K$）面前，底层算术指令的执行开销已经远远超过了 `malloc` 内存分配的系统开销。**

- **三重翻转法**：总共需要进行约 N 次 Swap 交换，每次 Swap 包含 2 次 `get` 和至少 0~2 次 `set`。总位读取次数固定为 $4N$ **次**。
- **缓冲交换法**：复制 K 位，移动 $N-K$ 位，写回 K 位。总位读写次数仅为 $N+K$ **次**。

最终决策：
由于位操作极其昂贵，我们最终在提交版中选择了减少 API 调用次数的“缓冲交换法”，并将这份三重翻转法代码作为重要的过程资产保留。